CS 224N Winter 2026 Assignment 2
Word2Vec and Dependency Parsing

Due date: January 22nd, Thursday, 4:30 PM PST
In this assignment, you will review the mathematics behind Word2Vec and build a neural dependency parser using PyTorch. For a review of the fundamentals of PyTorch, please check out the PyTorch review session on Canvas. In Part 1, you will explore the partial derivatives involved in training a Word2vec model using the naive softmax loss. In Part 2, you will learn about two general neural network techniques (Adam Optimization and Dropout). In Part 3, you will implement and train a dependency parser using the techniques from Part 2, before analyzing a few erroneous dependency parses.
If you are using LaTeX, you can use `\ifans{}` to type your solutions.
Please tag the questions correctly on Gradescope, otherwise the TAs will take points off if you don't tag questions.

1. Understanding word2vec (20 points)

Recall that the key insight behind `word2vec` is that *'a word is known by the company it keeps'*. Concretely, consider a 'center' word c surrounded before and after by a context of a certain length. We term words in this contextual window 'outside words' (O). For example, in Figure 1, the context window length is 2, the center word c is 'banking', and the outside words are 'turning', 'into', 'crises', and 'as':

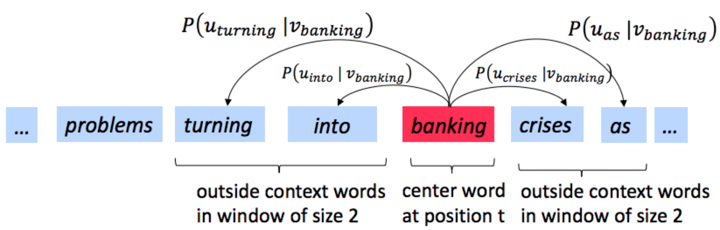


Figure 1: The word2vec skip-gram prediction model with window size 2

Skip-gram `word2vec` aims to learn the probability distribution $P(O|C)$. Specifically, given a specific word o and a specific word c, we want to predict $P(O = o|C = c)$: the probability that word o is an 'outside' word for c (i.e., that it falls within the contextual window of c). We model this probability by taking the softmax function over a series of vector dot-products:

$$P(O = o \mid C = c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \tag{1}$$

For each word, we learn vectors u and v, where $\mathbf{u}_o$ is the 'outside' vector representing outside word o, and $\mathbf{v}_c$ is the 'center' vector representing center word c. We store these parameters in two matrices, $\mathbf{U}$ and $\mathbf{V}$. The columns of $\mathbf{U}$ are all the 'outside' vectors $\mathbf{u}_w$; the columns of $\mathbf{V}$ are all of the 'center' vectors $\mathbf{v}_w$. Both $\mathbf{U}$ and $\mathbf{V}$ contain a vector for every $w \in$ Vocabulary.[1]

[1]Assume that every word in our vocabulary is matched to an integer number k. Bolded lowercase letters represent vectors. $\mathbf{u}_k$ is both the k^{th} column of $\mathbf{U}$ and the 'outside' word vector for the word indexed by k. $\mathbf{v}_k$ is both the k^{th} column of $\mathbf{V}$ and the 'center' word vector for the word indexed by k. **In order to simplify notation we shall interchangeably use k to refer to word k and the index of word k.**

Recall from lectures that, for a single pair of words c and o, the loss is given by:

$$\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U}) = -\log P(O = o|C = c). \tag{2}$$

We can view this loss as the cross-entropy[2] between the true distribution $\mathbf{y}$ and the predicted distribution $\hat{\mathbf{y}}$, for a particular center word c and a particular outside word o. Here, both $\mathbf{y}$ and $\hat{\mathbf{y}}$ are vectors with length equal to the number of words in the vocabulary. Furthermore, the k^{th} entry in these vectors indicates the conditional probability of the k^{th} word being an 'outside word' for the given c. The true empirical distribution $\mathbf{y}$ is a one-hot vector with a 1 for the true outside word o, and 0 everywhere else, for this particular example of center word c and outside word o.[3] The predicted distribution $\hat{\mathbf{y}}$ is the probability distribution $P(O|C = c)$ given by our model in equation (1).

Note: Throughout this homework, when computing derivatives, please use the method reviewed during the lecture (i.e. no Taylor Series Approximations).

(a) (2 points) Prove that the naive-softmax loss (Equation 2) is the same as the cross-entropy loss between $\mathbf{y}$ and $\hat{\mathbf{y}}$, i.e. (note that $\mathbf{y}$ (true distribution), $\hat{\mathbf{y}}$ (predicted distribution) are vectors and $\hat{\mathbf{y}}_o$ is a scalar):

$$-\sum_{w\in \text{Vocab}} \mathbf{y}_w \log(\hat{\mathbf{y}}_w) = -\log(\hat{\mathbf{y}}_o). \tag{3}$$

Your answer should be one line. You may describe your answer in words.

Solution: Since $\mathbf{y}$ is a one-hot vector where $\mathbf{y}_o = 1$ and $\mathbf{y}_w = 0$ for all $w \neq o$, the summation $\sum_{w\in \text{Vocab}} \mathbf{y}_w \log(\hat{\mathbf{y}}_w)$ collapses to only the term where $w = o$, which is $1 \cdot \log(\hat{\mathbf{y}}_o)$. Thus, the equality holds.

(b) (6 points) i. Compute the partial derivative of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to $\mathbf{v}_c$. *Please write your answer in terms of* $\mathbf{y}$, $\hat{\mathbf{y}}$, $\mathbf{U}$, *and show your work to receive full credit.*

Solution:

Substituting Equation 1 into Equation 2:

$$\begin{aligned} J &= -\log\left(\frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w\in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}\right) \\ &= -\mathbf{u}_o^\top \mathbf{v}_c + \log\left(\sum_{w\in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)\right) \end{aligned}$$

Taking the partial derivative with respect to $\mathbf{v}_c$:

$$\begin{aligned} \frac{\partial J}{\partial \mathbf{v}_c} &= \frac{\partial}{\partial \mathbf{v}_c}\left(-\mathbf{u}_o^\top \mathbf{v}_c\right) + \frac{\partial}{\partial \mathbf{v}_c} \log\left(\sum_{w\in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)\right) \\ &= -\mathbf{u}_o + \frac{1}{\sum_{w\in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \cdot \frac{\partial}{\partial \mathbf{v}_c}\left(\sum_{w\in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)\right) \\ &= -\mathbf{u}_o + \frac{\sum_{w\in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)\mathbf{u}_w}{\sum_{w\in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \\ &= -\mathbf{u}_o + \sum_{w\in \text{Vocab}} \left(\frac{\exp(\mathbf{u}_w^\top \mathbf{v}_c)}{\sum_{k\in \text{Vocab}} \exp(\mathbf{u}_k^\top \mathbf{v}_c)}\right) \mathbf{u}_w \\ &= -\mathbf{u}_o + \sum_{w\in \text{Vocab}} \hat{\mathbf{y}}_w \mathbf{u}_w \end{aligned}$$

[2] The **cross-entropy loss** between the true (discrete) probability distribution p and another distribution q is $-\sum_i p_i \log(q_i)$.
[3] Note that the true conditional probability distribution of context words for the entire training dataset would not be one-hot.

Using vectorized notation where $\mathbf{u}_o = \mathbf{U}\mathbf{y}$ and $\sum_w \hat{\mathbf{y}}_w \mathbf{u}_w = \mathbf{U}\hat{\mathbf{y}}$:

$$\frac{\partial J}{\partial \mathbf{v}_c} = \mathbf{U}(\hat{\mathbf{y}} - \mathbf{y})$$

- **Note**: Your final answers for the partial derivative should follow the shape convention: the partial derivative of any function $f(x)$ with respect to x should have the **same shape** as x.[4]
- Please provide your answers for the partial derivative in vectorized form. For example, when we ask you to write your answers in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{U}$, you may not refer to specific elements of these terms in your final answer (such as $\mathbf{y}_1$, $\mathbf{y}_2$, $\ldots$). You may also not refer to specific vectors such as u_0, u_1, etc.

ii. When is the gradient you computed equal to zero? Write a mathematical equation.
Hint: You may wish to review and use some introductory linear algebra concepts.
Solution:
The gradient is zero when $\hat{\mathbf{y}} = \mathbf{y}$.
Since $\hat{\mathbf{y}}$ is the predicted probability distribution and $\mathbf{y}$ is the ground-truth one-hot vector, this equality holds when the model's prediction is perfect, i.e., $P(O = o|C = c) = 1$ and $P(O = w|C = c) = 0$ for all $w \neq o$.

iii. The gradient you found is the difference between the two terms. Provide an interpretation of how each of these terms improves the word vector when this gradient is subtracted from the word vector v_c.
Solution:
The update rule for gradient descent is $\mathbf{v}_c \leftarrow \mathbf{v}_c - \eta \frac{\partial J}{\partial \mathbf{v}_c}$. Substituting our result:

$$\mathbf{v}_c \leftarrow \mathbf{v}_c + \eta(\mathbf{u}_o - \sum_{w \in \text{Vocab}} \hat{\mathbf{y}}_w \mathbf{u}_w)$$

The interpretation of the two terms is as follows:

(a) **The observed term $\mathbf{u}_o$**: Since the similarity between words is measured by their dot product, adding a fraction of $\mathbf{u}_o$ to $\mathbf{v}_c$ reduces the angle between these two vectors. This increases their dot product and makes $\mathbf{v}_c$ more similar to the actual outside word vector, thereby increasing the probability of the true context word.

(b) **The expectation term $\sum_w \hat{\mathbf{y}}_w \mathbf{u}_w$**: This term represents the "average" context word vector as predicted by the current model. By subtracting this from $\mathbf{v}_c$, we move $\mathbf{v}_c$ away from words that the model incorrectly assigns high probability to, effectively "de-biasing" the prediction.

(c) (1 point) In many downstream applications using word embeddings, L2 normalized vectors (e.g. $\mathbf{u}/||\mathbf{u}||_2$ where $||\mathbf{u}||_2 = \sqrt{\sum_i u_i^2}$) are used instead of their raw forms (e.g. $\mathbf{u}$). Let's consider a hypothetical downstream task of binary classification of phrases as being positive or negative, where you decide the sign based on the sum of individual embeddings of the words. When would L2 normalization take away useful information for the downstream task? When would it not?

Hint: Consider the case where $\mathbf{u}_x = \alpha \mathbf{u}_y$ for some words $x \neq y$ and some scalar α. When α is positive, what will be the value of normalized $\mathbf{u}_x$ and normalized $\mathbf{u}_y$? How might $\mathbf{u}_x$ and $\mathbf{u}_y$ be related for such a normalization to affect or not affect the resulting classification?

Solution:

[4] This allows us to efficiently minimize a function using gradient descent without worrying about reshaping or dimension mismatching. While following the shape convention, we're guaranteed that $\theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$ is a well-defined update rule.

L2 normalization takes away useful information when the magnitude of the word vector represents its semantic intensity or importance. For example, if $\mathbf{u}_{excellent} = \alpha \mathbf{u}_{good}$ with $\alpha > 1$, normalization would make them identical, losing the information that "excellent" is a stronger positive signal than "good" in a sum of embeddings.

It would not take away useful information (and might even help) if the magnitude mainly reflects word frequency or noise rather than meaning. In such cases, the direction of the vector captures the core semantics, and normalization prevents high-frequency words from disproportionately dominating the classification result.

(d) (1 point) Write down the partial derivative of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to $\mathbf{U}$. Please break down your answer in terms of the column vectors $\frac{\partial \mathbf{J}(\mathbf{v}_c,o,\mathbf{U})}{\partial \mathbf{u}_1}$, $\frac{\partial \mathbf{J}(\mathbf{v}_c,o,\mathbf{U})}{\partial \mathbf{u}_2}$, $\cdots$, $\frac{\partial \mathbf{J}(\mathbf{v}_c,o,\mathbf{U})}{\partial \mathbf{u}_{|\text{Vocab}|}}$ (do not further expand these terms). No derivations are necessary, just an answer in the form of a matrix.

Solution:

$$\frac{\partial J}{\partial \mathbf{U}} = \begin{bmatrix} \frac{\partial J}{\partial \mathbf{u}_1} & \frac{\partial J}{\partial \mathbf{u}_2} & \cdots & \frac{\partial J}{\partial \mathbf{u}_{|\text{Vocab}|}} \end{bmatrix}$$

(e) (5 points) Compute the partial derivatives of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to each of the 'outside' word vectors, $\mathbf{u}_w$'s. There will be two cases: when $w = o$, the true 'outside' word vector, and $w \neq o$, for all other words. Please write your answer in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{v}_c$. In this subpart, you may use specific elements within these terms as well (such as $\mathbf{y}_1$, $\mathbf{y}_2$, $\ldots$). Note that $\mathbf{u}_w$ is a vector while $\mathbf{y}_1, \mathbf{y}_2, \ldots$ are scalars. Show your work to receive full credit.

Solution:

The loss function is $J = -\mathbf{u}_o^\top \mathbf{v}_c + \log \sum_{k \in \text{Vocab}} \exp(\mathbf{u}_k^\top \mathbf{v}_c)$.

Case 1: $w = o$

$$\begin{aligned} \frac{\partial J}{\partial \mathbf{u}_o} &= \frac{\partial}{\partial \mathbf{u}_o}\left(-\mathbf{u}_o^\top \mathbf{v}_c\right) + \frac{\partial}{\partial \mathbf{u}_o} \log\left(\sum_{k \in \text{Vocab}} \exp(\mathbf{u}_k^\top \mathbf{v}_c)\right) \\ &= -\mathbf{v}_c + \frac{1}{\sum_{k \in \text{Vocab}} \exp(\mathbf{u}_k^\top \mathbf{v}_c)} \cdot \exp(\mathbf{u}_o^\top \mathbf{v}_c)\mathbf{v}_c \\ &= -\mathbf{v}_c + \hat{y}_o \mathbf{v}_c = (\hat{y}_o - 1)\mathbf{v}_c \end{aligned}$$

Case 2: $w \neq o$

$$\begin{aligned} \frac{\partial J}{\partial \mathbf{u}_w} &= 0 + \frac{\partial}{\partial \mathbf{u}_w} \log\left(\sum_{k \in \text{Vocab}} \exp(\mathbf{u}_k^\top \mathbf{v}_c)\right) \\ &= \frac{1}{\sum_{k \in \text{Vocab}} \exp(\mathbf{u}_k^\top \mathbf{v}_c)} \cdot \exp(\mathbf{u}_w^\top \mathbf{v}_c)\mathbf{v}_c \\ &= \hat{y}_w \mathbf{v}_c \end{aligned}$$

Since $y_o = 1$ and $y_w = 0$ for $w \neq o$, both cases can be summarized as:

$$\frac{\partial J}{\partial \mathbf{u}_w} = (\hat{y}_w - y_w)\mathbf{v}_c$$

(f) (2 points) As an additional exercise for this problem, you will be taking the derivatives of some common loss functions, which may be used in variations of `word2vec` (such as the negative sampling variant). The Leaky ReLU (Leaky Rectified Linear Unit) activation function is given by Equation 4 and Figure 2:

$$f(x) = \max(\alpha x, x) \tag{4}$$

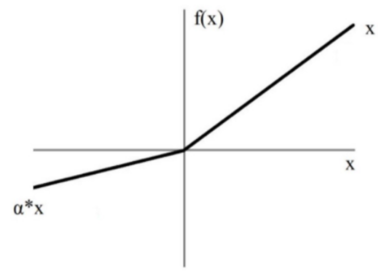


Figure 2: Leaky ReLU

Where x is a scalar and $0 < \alpha < 1$, please compute the derivative of $f(x)$ with respect to x. You may ignore the case where the derivative is not defined at 0.[5]

Solution: For $x \neq 0$, the derivative of $f(x) = \max(\alpha x, x)$ is:

$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{if } x < 0 \end{cases}$$

(g) (3 points) The sigmoid function is given by Equation 5:

$$\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{e^x+1} \tag{5}$$

Please compute the derivative of $\sigma(x)$ with respect to x, where x is a scalar. Please write your answer in terms of $\sigma(x)$. Show your work to receive full credit.

Solution: Using the chain rule:

$$\begin{aligned}
\sigma'(x) &= \frac{d}{dx}(1+e^{-x})^{-1} \\
&= -1 \cdot (1+e^{-x})^{-2} \cdot \frac{d}{dx}(1+e^{-x}) \\
&= -(1+e^{-x})^{-2} \cdot (-e^{-x}) \\
&= \frac{e^{-x}}{(1+e^{-x})^2} \\
&= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} \\
&= \sigma(x) \cdot \frac{1+e^{-x}-1}{1+e^{-x}} \\
&= \sigma(x) \cdot (1-\sigma(x))
\end{aligned}$$

[5] If you're interested in how to handle the derivative at this point, you can read more about the notion of subderivatives.

2. Neural Networks Optimization (8 points)

(a) (4 points) Adam Optimizer

Recall the standard Stochastic Gradient Descent update rule:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t)$$

where $t+1$ is the current timestep, $\boldsymbol{\theta}$ is a vector containing all of the model parameters, ($\boldsymbol{\theta}_t$ is the model parameter at time step t, and $\boldsymbol{\theta}_{t+1}$ is the model parameter at time step $t+1$), J is the loss function, $\nabla_{\boldsymbol{\theta}} J_{\text{minibatch}}(\boldsymbol{\theta})$ is the gradient of the loss function with respect to the parameters on a minibatch of data, and α is the learning rate. Adam Optimization[6] uses a more sophisticated update rule with two additional steps.[7]

i. (2 points) First, Adam uses a trick called *momentum* by keeping track of $\mathbf{m}$, a rolling average of the gradients:

$$\mathbf{m}_{t+1} \leftarrow \beta_1 \mathbf{m}_t + (1-\beta_1)\nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t)$$
$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \mathbf{m}_{t+1}$$

where β_1 is a hyperparameter between 0 and 1 (often set to 0.9). **Briefly explain in 2–4 sentences** (you don't need to prove mathematically, just give an intuition) how using $\mathbf{m}$ stops the updates from varying as much and why this low variance may be helpful to learning, overall.

Solution: Momentum acts as a rolling average that smooths out high-frequency oscillations in the gradients by canceling out opposing directions and amplifying consistent ones. This reduces the variance of the updates, which is particularly helpful in navigating narrow "ravines" in the loss surface where gradients might otherwise oscillate between steep walls. Overall, this leads to more stable and faster convergence towards the minimum.

ii. (2 points) Adam extends the idea of *momentum* with the trick of *adaptive learning rates* by keeping track of $\mathbf{v}$, a rolling average of the magnitudes of the gradients:

$$\mathbf{m}_{t+1} \leftarrow \beta_1 \mathbf{m}_t + (1-\beta_1)\nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t)$$
$$\mathbf{v}_{t+1} \leftarrow \beta_2 \mathbf{v}_t + (1-\beta_2)(\nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t) \odot \nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t))$$
$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \mathbf{m}_{t+1} / \sqrt{\mathbf{v}_{t+1}}$$

where $\odot$ and $/$ denote elementwise multiplication and division (so $\mathbf{z} \odot \mathbf{z}$ is elementwise squaring) and β_2 is a hyperparameter between 0 and 1 (often set to 0.99). Since Adam divides the update by $\sqrt{\mathbf{v}}$, which of the model parameters will get larger updates? Why might this help with learning? **Briefly explain in 2–4 sentences**.

Solution: Parameters with smaller gradient magnitudes (smaller partial derivatives) will receive larger updates because they are divided by a smaller denominator $\sqrt{\mathbf{v}}$. This helps with learning by ensuring that sparse or infrequent features, which typically have weak signals, can still make significant progress. Simultaneously, it prevents parameters with very large gradients from causing unstable updates or overshooting, leading to more robust optimization across all parameters.

(b) (4 points) Dropout[8] is a regularization technique. During training, dropout randomly sets units in the hidden layer $\mathbf{h}$ to zero with probability p_{drop} (dropping different units each minibatch), and then multiplies $\mathbf{h}$ by a constant γ. We can write this as:

$$\mathbf{h}_{\text{drop}} = \gamma \mathbf{d} \odot \mathbf{h}$$

[6] Kingma and Ba, 2015, `https://arxiv.org/pdf/1412.6980.pdf`

[7] The actual Adam update uses a few additional tricks that are less important, but we won't worry about them here. If you want to learn more about it, you can take a look at: `http://cs231n.github.io/neural-networks-3/#sgd`

[8] Srivastava et al., 2014, `https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf`

where $\mathbf{d} \in \{0,1\}^{D_h}$ (D_h is the size of $\mathbf{h}$) is a mask vector where each entry is 0 with probability p_{drop} and 1 with probability $(1 - p_{\text{drop}})$. γ is chosen such that the expected value of $\mathbf{h}_{\text{drop}}$ is $\mathbf{h}$:

$$\mathbb{E}_{p_{\text{drop}}}[\mathbf{h}_{\text{drop}}]_i = h_i$$

for all $i \in \{1, \ldots, D_h\}$.

i. (2 points) What must γ equal in terms of p_{drop}? Briefly justify your answer or show your math derivation using the equations given above.

Solution: To ensure the expected value of the dropped-out hidden layer remains consistent with the original values, we set the constraint $\mathbb{E}_{p_{\text{drop}}}[\gamma d_i h_i] = h_i$. Since γ and h_i are constants with respect to the mask variable d_i, we can apply the linearity of expectation:

$$\gamma h_i \mathbb{E}[d_i] = h_i \implies \gamma \mathbb{E}[d_i] = 1$$

Given that d_i is 0 with probability p_{drop} and 1 with probability $(1 - p_{\text{drop}})$, its expectation is:

$$\mathbb{E}[d_i] = (1 \cdot (1 - p_{\text{drop}})) + (0 \cdot p_{\text{drop}}) = 1 - p_{\text{drop}}$$

Substituting this back into the equation:

$$\gamma(1 - p_{\text{drop}}) = 1 \implies \gamma = \frac{1}{1 - p_{\text{drop}}}$$

ii. (2 points) Why should dropout be applied during training? Why should dropout **NOT** be applied during evaluation? **Briefly explain in 2–4 sentences**. **Hint:** it may help to look at the dropout paper linked.

Solution: During training, dropout is used to prevent neurons from relying too heavily on specific other units (breaking co-adaptation), which forces the network to learn more robust and distributed features, effectively mitigating overfitting. During evaluation, we disable dropout to utilize the full capacity of the trained network and ensure deterministic predictions, as we no longer need the stochastic regularization used during training.

3. Neural Transition-Based Dependency Parsing (40 points)

In this section, you'll be implementing a neural-network based dependency parser with the goal of maximizing performance on the UAS (Unlabeled Attachment Score) metric.

Before you begin, please follow the README to install all the needed dependencies for the assignment. We will be using PyTorch 2.1.2 from `https://pytorch.org/get-started/locally/` with the `CUDA` option set to `None`, and the tqdm package – which produces progress bar visualizations throughout your training process. The official PyTorch website is a great resource that includes tutorials for understanding PyTorch's Tensor library and neural networks.

A dependency parser analyzes the grammatical structure of a sentence, establishing relationships between *head* words, and words which modify those heads. There are multiple types of dependency parsers, including transition-based parsers, graph-based parsers, and feature-based parsers. Your implementation will be a *transition-based* parser, which incrementally builds up a parse one step at a time. At every step it maintains a *partial parse*, which is represented as follows:

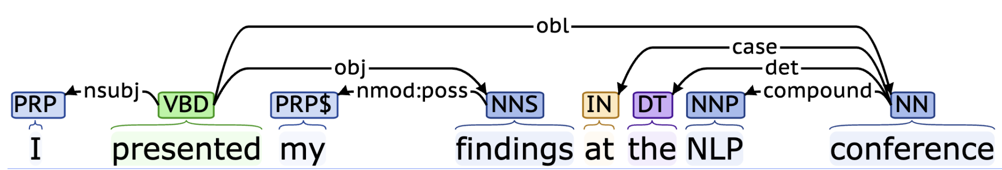
- A *stack* of words that are currently being processed.
- A *buffer* of words yet to be processed.
- A list of *dependencies* predicted by the parser.

Initially, the stack only contains ROOT, the dependencies list is empty, and the buffer contains all words of the sentence in order. At each step, the parser applies a *transition* to the partial parse until its buffer is empty and the stack size is 1. The following transitions can be applied:

- `SHIFT`: removes the first word from the buffer and pushes it onto the stack.
- `LEFT-ARC`: marks the second (second most recently added) item on the stack as a dependent of the first item and removes the second item from the stack, adding a *first_word* → *second_word* dependency to the dependency list.
- `RIGHT-ARC`: marks the first (most recently added) item on the stack as a dependent of the second item and removes the first item from the stack, adding a *second_word* → *first_word* dependency to the dependency list.

On each step, your parser will decide among the three transitions using a neural network classifier.

(a) (4 points) Go through the sequence of transitions needed for parsing the sentence *"I presented my findings at the NLP conference"*. The dependency tree for the sentence is shown below. At each step, give the configuration of the stack and buffer, as well as what transition was applied this step and what new dependency was added (if any). The first three steps are provided below as an example.



Stack	Buffer	New dependency	Tr
[ROOT]	[I, presented, my, findings, at, the, NLP, conference]		In
[ROOT, I]	[presented, my, findings, at, the, NLP, conference]		SH
[ROOT, I, presented]	[my, findings, at, the, NLP, conference]		SH
[ROOT, presented]	[my, findings, at, the, NLP, conference]	presented→I	LE
[ROOT, presented, my]	[findings, at, the, NLP, conference]		SH
[ROOT, presented, my, findings]	[at, the, NLP, conference]		SH
[ROOT, presented, findings]	[at, the, NLP, conference]	findings→my	LE
[ROOT, presented]	[at, the, NLP, conference]	presented→findings	RI
[ROOT, presented, at]	[the, NLP, conference]		SH
[ROOT, presented, at, the]	[NLP, conference]		SH
[ROOT, presented, at, the, NLP]	[conference]		SH
[ROOT, presented, at, the, NLP, conference]	[]		SH
[ROOT, presented, at, the, conference]	[]	conference→NLP	LE
[ROOT, presented, at, conference]	[]	conference→the	LE
[ROOT, presented, at]	[]	at→conference	RI
[ROOT, presented]	[]	presented→at	RI
[ROOT]	[]	ROOT→presented	RI

(b) (2 points) A sentence containing n words will be parsed in how many steps (in terms of n)? Briefly explain in 1–2 sentences why.

Solution: A sentence with n words will take $2n$ steps to be parsed. This is because each of the n words must be shifted onto the stack exactly once (n steps), and each of the n words must be removed from the stack through an ARC transition (either LEFT-ARC or RIGHT-ARC) exactly once to form a dependency (n steps).

(c) (6 points) Implement the `__init__` and `parse_step` functions in the `PartialParse` class in `parser_transitions.py`. This implements the transition mechanics your parser will use. You can run basic (non-exhaustive) tests by running `python parser_transitions.py part_c`.

(d) (8 points) Our network will predict which transition should be applied next to a partial parse. We could use it to parse a single sentence by applying predicted transitions until the parse is complete. However, neural networks run much more efficiently when making predictions about *batches* of data at a time (i.e., predicting the next transition for any different partial parses simultaneously). We can parse sentences in minibatches with the following algorithm.

Algorithm 1 Minibatch Dependency Parsing

Input: `sentences`, a list of sentences to be parsed and `model`, our model that makes parse decisions

Initialize `partial_parses` as a list of PartialParses, one for each sentence in `sentences`
Initialize `unfinished_parses` as a shallow copy of `partial_parses`
while `unfinished_parses` is not empty **do**
  Take the first `batch_size` parses in `unfinished_parses` as a minibatch
  Use the `model` to predict the next transition for each partial parse in the minibatch
  Perform a parse step on each partial parse in the minibatch with its predicted transition
  Remove the completed (empty buffer and stack of size 1) parses from `unfinished_parses`
end while

Return: The `dependencies` for each (now completed) parse in `partial_parses`.

Implement this algorithm in the `minibatch_parse` function in `parser_transitions.py`. You can run basic (non-exhaustive) tests by running `python parser_transitions.py part_d`.

Note: You will need `minibatch_parse` to be correctly implemented to evaluate the model you will build in part (e). However, you do not need it to train the model, so you should be able to complete most of part (e) even if `minibatch_parse` is not implemented yet.

(e) (20 points) We are now going to train a neural network to predict, given the state of the stack, buffer, and dependencies, which transition should be applied next.

First, the model extracts a feature vector representing the current state. We will be using the feature set presented in the original neural dependency parsing paper: *A Fast and Accurate Dependency Parser using Neural Networks*.[9] The function extracting these features has been implemented for you in `utils/parser_utils.py`. This feature vector consists of a list of tokens (e.g., the last word in the stack, first word in the buffer, dependent of the second-to-last word in the stack if there is one, etc.). They can be represented as a list of integers $\mathbf{w} = [w_1, w_2, \dots, w_m]$ where m is the number of features and each $0 \leq w_i < |V|$ is the index of a token in the vocabulary ($|V|$ is the vocabulary size). Then our network looks up an embedding for each word and concatenates them into a single input vector:

$$\mathbf{x} = [\mathbf{E}_{w_1}, ..., \mathbf{E}_{w_m}] \in \mathbb{R}^{dm}$$

where $\mathbf{E} \in \mathbb{R}^{|V| \times d}$ is an embedding matrix with each row $\mathbf{E}_w$ as the vector for a particular word w with dimension d. We then compute our prediction as:

$$\begin{aligned}
\mathbf{h} &= \mathrm{ReLU}(\mathbf{xW} + \mathbf{b}_1) \\
\mathbf{l} &= \mathbf{hU} + \mathbf{b}_2 \\
\hat{\mathbf{y}} &= \mathrm{softmax}(l)
\end{aligned}$$

where $\mathbf{h}$ is referred to as the hidden layer, $\mathbf{l}$ is referred to as the logits, $\hat{\mathbf{y}}$ is referred to as the predictions, and $\mathrm{ReLU}(z) = \max(z, 0)$). We will train the model to minimize cross-entropy loss:

$$J(\theta) = CE(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^{3} \mathbf{y}_j \log \hat{\mathbf{y}}_j$$

where $\mathbf{y}_j$ denotes the jth element of $\mathbf{y}$. To compute the loss for the training set, we average this $J(\theta)$ across all training examples.

i. Compute the derivative of $\mathbf{h} = \mathrm{ReLU}(\mathbf{xW} + \mathbf{b}_1)$ with respect to $\mathbf{x}$. For simplicity, you only need to show the derivative $\frac{\partial h_i}{\partial x_j}$ for some index i and j. You may ignore the case where the derivative is not defined at 0.

Solution: Let $\mathbf{z} = \mathbf{xW} + \mathbf{b}_1$, so that $h_i = \mathrm{ReLU}(z_i)$. By the chain rule:

$$\frac{\partial h_i}{\partial x_j} = \frac{\partial h_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial x_j}$$

The derivative of the ReLU function is:

$$\frac{\partial h_i}{\partial z_i} = \begin{cases} 1 & \text{if } z_i > 0 \\ 0 & \text{if } z_i < 0 \end{cases}$$

[9] Chen and Manning, 2014, `https://nlp.stanford.edu/pubs/emnlp2014-depparser.pdf`

The linear term $z_i = \sum_k x_k W_{ki} + (b_1)_i$ gives:

$$\frac{\partial z_i}{\partial x_j} = W_{ji}$$

Thus, the partial derivative is:

$$\frac{\partial h_i}{\partial x_j} = \begin{cases} W_{ji} & \text{if } z_i > 0 \\ 0 & \text{if } z_i < 0 \end{cases}$$

ii. Recall in part 1b, we computed the partial derivative of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$. Likewise, please compute the partial derivative of $J(\theta)$ with respect to the ith entry of $\mathbf{l}$, which is denoted as $\mathbf{l}_i$. Specifically, compute $\frac{\partial CE(\mathbf{y},\hat{\mathbf{y}})}{\partial \mathbf{l}_i}$, assuming that $\mathbf{l} \in \mathbb{R}^3$, $\hat{\mathbf{y}} \in \mathbb{R}^3$, $\mathbf{y} \in \mathbb{R}^3$, and the true label is c (i.e., $y_j = 1$ if $j = c$). **Hint**: Use the chain rule: $\frac{\partial J}{\partial \mathbf{l}} = \frac{\partial J}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{l}}$.
Solution: From the derivation in 1(b) and 1(e), the derivative of the cross-entropy loss with respect to the logits of a softmax function is:

$$\frac{\partial J}{\partial l_i} = \hat{y}_i - y_i$$

Given the true label is c ($y_c = 1$ and $y_j = 0$ for $j \neq c$), the result is:

$$\frac{\partial J}{\partial l_i} = \begin{cases} \hat{y}_i - 1 & \text{if } i = c \\ \hat{y}_i & \text{if } i \neq c \end{cases}$$

iii. We will use UAS score as our evaluation metric. UAS refers to Unlabeled Attachment Score, which is computed as the ratio between number of correctly predicted dependencies and the number of total dependencies despite of the relations (our model doesn't predict this).

In `parser_model.py` you will find skeleton code to implement this simple neural network using PyTorch. Complete the `__init__`, `embedding_lookup` and `forward` functions to implement the model. Then complete the `train_for_epoch` and `train` functions within the `run.py` file.
Finally execute `python run.py` to train your model and compute predictions on test data from Penn Treebank (annotated with Universal Dependencies).
Note:

- For this assignment, you are asked to implement Linear layer and Embedding layer. Please **DO NOT** use **torch.nn.Linear** or **torch.nn.Embedding** module in your code, otherwise you will receive deductions for this problem.
- Please follow the naming requirements in our TODO if there are any, e.g. if there are explicit requirements about variable names you have to follow them in order to receive full credits. You are free to declare other variable names if not explicitly required.

Hints:

- Each of the variables you are asked to declare (`self.embed_to_hidden_weight`, `self.embed_to_hidden_bias`, `self.hidden_to_logits_weight`, `self.hidden_to_logits_bias`) corresponds to one of the variables above ($\mathbf{W}$, $\mathbf{b}_1$, $\mathbf{U}$, $\mathbf{b}_2$).
- It may help to work backwards in the algorithm (start from $\hat{\mathbf{y}}$) and keep track of the matrix/vector sizes.
- Once you have implemented `embedding_lookup (e)` or `forward (f)` you can call `python parser_model.py` with flag `-e` or `-f` or both to run sanity checks with each

function. These sanity checks are fairly basic and passing them doesn't mean your code is bug free.

- When debugging, you can add a debug flag: `python run.py -d`. This will cause the code to run over a small subset of the data, so that training the model won't take as long. Make sure to remove the `-d` flag to run the full model once you are done debugging.
- When running with debug mode, you should be able to get a loss smaller than 0.2 and a UAS larger than 65 on the dev set (although in rare cases your results may be lower, there is some randomness when training).
- It should take up to **15 minutes** to train the model on the entire training dataset, i.e., when debug mode is disabled.
- When debug mode is disabled, you should be able to get a loss smaller than 0.08 on the train set and an Unlabeled Attachment Score larger than 87 on the dev set. For comparison, the model in the original neural dependency parsing paper gets 92.5 UAS. If you want, you can tweak the hyperparameters for your model (hidden layer size, hyperparameters for Adam, number of epochs, etc.) to improve the performance (but you are not required to do so).

Deliverables:

- Working implementation of the transition mechanics that the neural dependency parser uses in `parser_transitions.py`.
- Working implementation of minibatch dependency parsing in `parser_transitions.py`.
- Working implementation of the neural dependency parser in `parser_model.py`. (We'll look at and run this code for grading).
- Working implementation of the functions for training in `run.py`. (We'll look at and run this code for grading).
- Please use efficient functions and **avoid for loops** when implementing `embedding_lookup`. **Otherwise, you may exceed the GradeScope test time limit.**
- **Report the best UAS your model achieves on the dev set and the UAS it achieves on the test set in your written submission**. You can report it in the PDF and tag the page.

Submission Instructions

You shall submit this assignment on GradeScope as two submissions – one for `Assignment 2 [coding]` and another for `Assignment 2 [written]`:

1. Run the `collect_submission.sh` script to produce your `assignment2.zip` file.
2. Upload your `assignment2.zip` file to GradeScope to `Assignment 2 [coding]`.
3. Upload your written solutions to GradeScope to `Assignment 2 [written]`.